Relatório Acadêmico

Sistema de Locadora de Veículos

Trabalho Prático

Tecnologias para Análise e Desenvolvimento de Sistemas

Marco Túlio

Vídeo de apresentação

https://youtu.be/-LkULsgQMv0

1. Vídeo de apresentação

A demonstração do sistema em funcionamento está hospedada no YouTube, com duração de **11 minutos e 29 segundos**, em resolução 1920×1080. O vídeo está publicado como **não listado**: fica acessível a qualquer pessoa que possua o endereço, sem necessidade de autenticação, e permanecerá disponível durante todo o período de avaliação.

https://youtu.be/-LkULsgQMv0

1.1 Índice do vídeo

Os tempos abaixo permitem ir diretamente a cada trecho.

Tempo	Trecho	Conteúdo
0:00	Abertura e modelagem	Apresentação do sistema; as cinco entidades no código; a classe ApplicationContext; as tabelas, chaves primárias e estrangeiras no SQL Server Express
2:05	Cadastro de veículos	Consulta da frota com fabricante e categoria; cadastro de um novo veículo retornando 201 Created
3:27	Locação e devolução	Registro de um aluguel com diária herdada da categoria; baixa do veículo na frota; devolução com cálculo de dias, valor total e quilometragem rodada
5:57	Consultas com junção	Quatro das seis rotas de filtro, demonstrando INNER JOIN e LEFT JOIN
8:31	Validação e tratamento de erros	Duplicidade (409), dados inválidos (400) e exclusão de registro com dependentes (409)
10:43	Encerramento	Síntese do que foi entregue

2. Visão geral do sistema

O sistema é uma API RESTful para gestão de uma locadora de veículos, construída em ASP.NET Core 8 com Entity Framework Core no padrão Code First e persistência em SQL Server Express. A documentação interativa é gerada pelo Swagger.

A API cobre o ciclo completo da operação: cadastro de fabricantes, categorias, veículos e clientes; registro de locações; devolução com cálculo automático de valores; e consultas gerenciais que combinam múltiplas tabelas.

Camada	Tecnologia
Linguagem	C# 12 sobre .NET 8
API	ASP.NET Core Web API, com controllers
Acesso a dados	Entity Framework Core 8 (Code First, com migrations)
Banco de dados	SQL Server Express 2025 — instância SQLEXPRESS
Documentação	Swagger / Swashbuckle, alimentado por comentários XML

Camada	Tecnologia
Controle de versão	Git e GitHub

2.1 Modelo de dados

São cinco entidades. A chave estrangeira reside sempre do lado "muitos" do relacionamento.

Entidade	Chave primária	Chaves estrangeiras	Papel
Fabricante	Id	—	Marcas dos veículos
CategoriaVeiculo	Id	—	Classes de veículo e tabela de diárias
Veiculo	Id	FabricanteId, CategoriaVeiculoId	A frota
Cliente	Id	—	Locatários
Aluguel	Id	ClienteId, VeiculoId	Locações e devoluções

Três campos do Aluguel são anuláveis por decisão de modelagem — data de devolução real, quilometragem final e valor total. Eles permanecem vazios enquanto o veículo não retorna, e é essa ausência que identifica as locações em aberto.

O valor da diária é cadastrado na categoria, mas copiado para o aluguel no momento da locação. Assim, reajustes na tabela de preços não alteram o valor de contratos já firmados.

3. Backend

São **32 endpoints** distribuídos em seis controllers. Cinco oferecem as operações CRUD completas de cada entidade; o sexto reúne as consultas analíticas.

Controller	Endpoints	Observação
Fabricantes	5	CRUD completo
CategoriasVeiculo	5	CRUD completo
Veiculos	5	CRUD completo, com fabricante e categoria carregados nas consultas
Clientes	5	CRUD completo
Alugueis	6	CRUD completo mais o endpoint de registro de devolução
Filtros	6	Consultas com junção de tabelas

3.1 Validação e tratamento de erros

A validação ocorre em três camadas independentes, de modo que uma falha em qualquer nível produza uma resposta compreensível em vez de um erro genérico.

Camada	Mecanismo	Protege contra
Modelo	Anotações de dados verificadas via ModelState	Campos obrigatórios ausentes, limites de tamanho, faixas numéricas e formato de e-mail
Regra de negócio	Verificações no controller antes da gravação	Duplicidade de placa, CPF e nomes; chaves estrangeiras inexistentes; datas e quilometragens incoerentes; veículo já locado; exclusão de registros com dependentes
Banco de dados	Índices únicos e OnDelete Restrict	Última barreira de integridade referencial

Exceções não previstas são interceptadas por um middleware global, registradas no log da aplicação e devolvidas como JSON de status 500, sem expor a pilha de execução.

3.2 Consultas com junção de tabelas

Seis rotas analíticas, escritas em sintaxe de consulta LINQ. Quatro empregam INNER JOIN e duas empregam LEFT JOIN.

Rota	Junção	Retorna
/api/Filtros/veiculos	INNER JOIN	Frota com marca e categoria, com filtros opcionais
/api/Filtros/clientes-e-alugueis	LEFT JOIN	Todos os clientes, inclusive os sem locação
/api/Filtros/alugueis-em-aberto	INNER JOIN	Locações não devolvidas, com cálculo de atraso
/api/Filtros/faturamento-por-categoria	INNER JOIN	Receita e quilometragem agrupadas por categoria
/api/Filtros/veiculos-nunca-alugados	LEFT JOIN	Veículos que nunca foram locados

Rota	Junção	Retorna
/api/Filtros/devolucoes-em-atraso	INNER JOIN	Devoluções entregues após o prazo

4. Documentação e testes

Dois documentos complementam este relatório, ambos na pasta **Docs** do repositório.

Documento	Conteúdo
Documentacao_Endpoints_API.pdf	Os 32 endpoints, com métodos HTTP, parâmetros, corpo da requisição e códigos de resposta
Relatorio_Testes_API.pdf	38 testes manuais executados via Swagger, cada um com objetivo, resultado esperado e captura de tela da chamada com o retorno obtido

5. Repositório

O código-fonte, as migrations, os scripts SQL, a documentação e as evidências de teste estão versionados em:

https://github.com/MarcoTulioGr/LocadoraVeiculos

Para executar o projeto: restaurar os pacotes NuGet, ajustar a string de conexão em **appsettings.json** conforme a instância local do SQL Server Express, executar **Update-Database** no Console do Gerenciador de Pacotes e, opcionalmente, rodar o script **Scripts/01_povoamento.sql** para carregar a massa de dados usada na demonstração.